Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 1m

```
RATE = 40.0
DAYS = 10

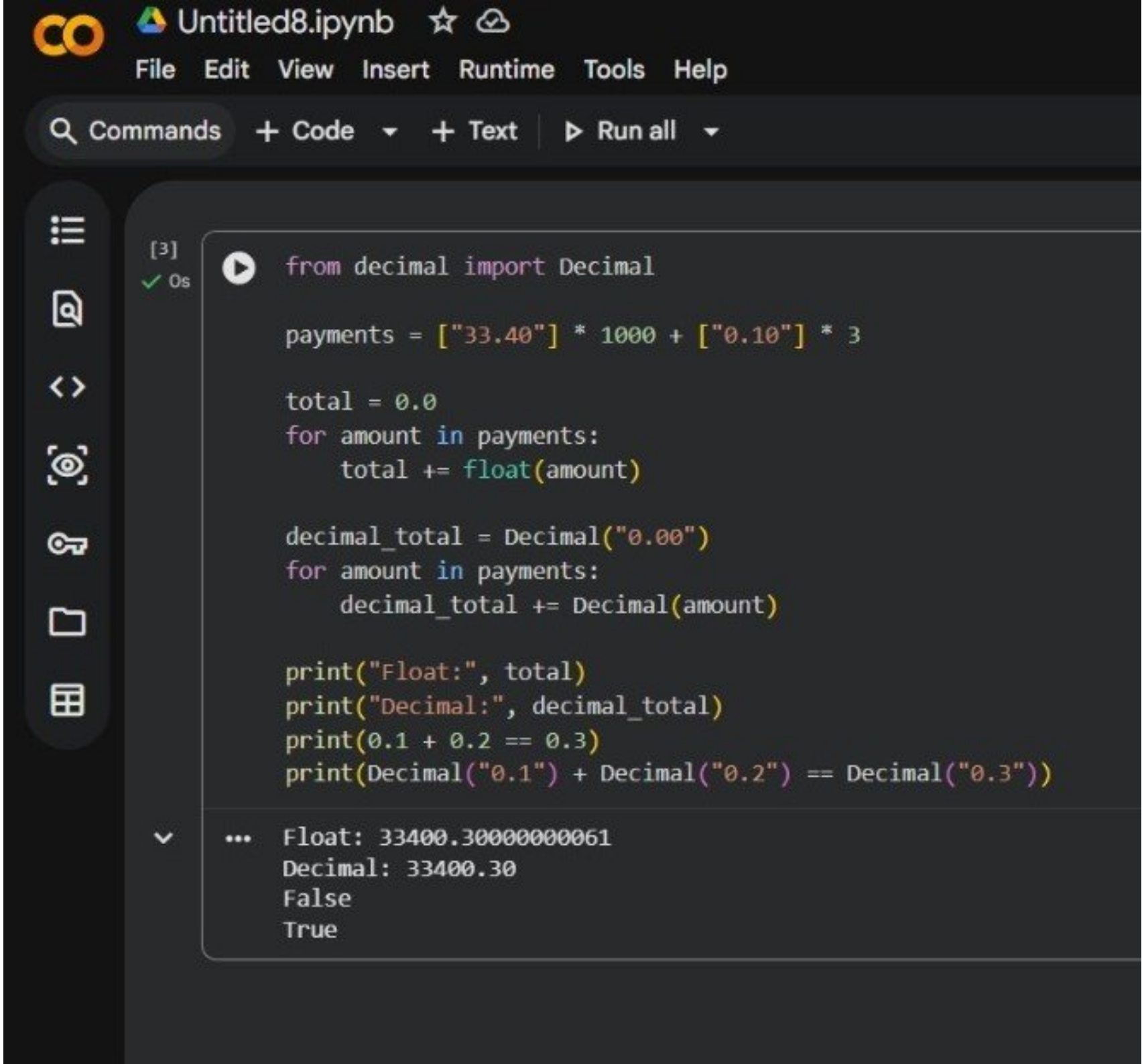
farmer_id = input("Farmer ID: ")
litres = float(input("Litres: "))
fat = float(input("Fat %: "))
snf = float(input("SNF %: "))
shift = input("Shift: ")

if litres <= 0 or not 2 <= fat <= 7:
    print("Invalid input")
else:
    rate = RATE + fat * 2 + snf
    amount = litres * rate
    total = amount * DAYS

    print("Rate per litre:", rate)
    print("Payment amount:", amount)
    print("Ten-day total:", total)
```

```
Farmer ID: S107
Litres: 20
Fat %: 4
SNF %: 8
Shift: Morning
Rate per litre: 56.0
Payment amount: 1120.0
Ten-day total: 11200.0
```

Variables Terminal


Untitled8.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[3]
0s
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3
total = 0.0
for amount in payments:
total += float(amount)
decimal_total = Decimal("0.00")
for amount in payments:
decimal_total += Decimal(amount)
print("Float:", total)
print("Decimal:", decimal_total)
print(0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
Float: 33400.30000000061
Decimal: 33400.30
False
True

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[4]
✓ 0s

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

fat = 4.0
snf = 8.0

fat_steps = fat * 10
snf_steps = snf * 10

rate = BASE_RATE + fat * FAT_RATE + snf * SNF_RATE
amount = 10 * rate

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", rate)
print("Payment amount:", amount)

a = 10
b = 20
a, b = b, a
```

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Fat steps: 40.0
SNF steps: 80.0
Rate per litre: 34.0
Payment amount: 340.0
Swapped: 20 10
```

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[6]
✓ 0s

```
fat = 4.2
snf = 3.1

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 3.0) / 0.1)

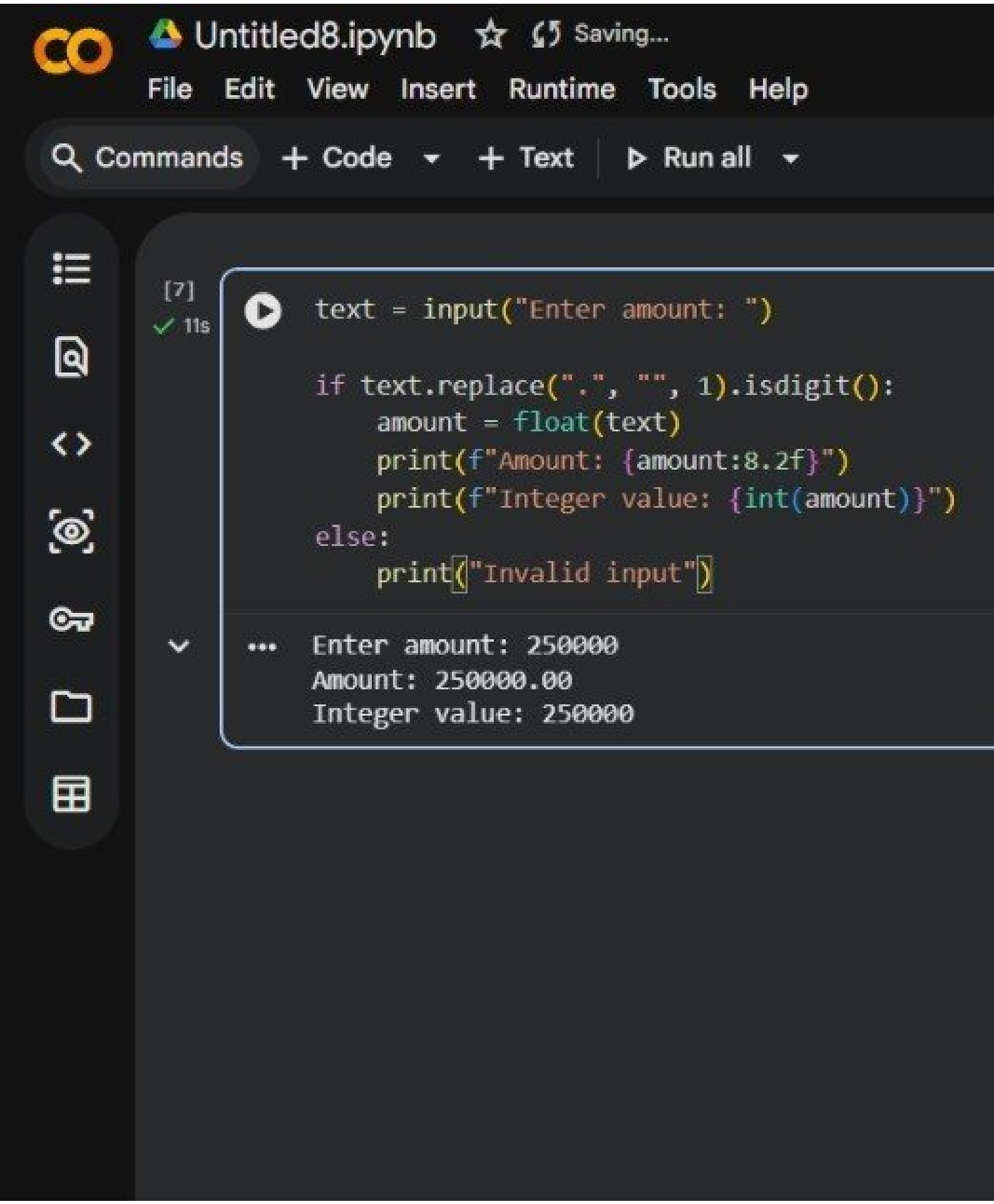
rate = 30.0 + fat_steps * 0.40 + snf_steps * 0.30
amount = 10 * rate

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", rate)
print("Amount:", amount)

print((4.2 - 3.5) / 0.1)
print(round((4.2 - 3.5) / 0.1))
print(int((4.2 - 3.5) / 0.1))
print(2.0 <= fat <= 7.0)
print(0.1 + 0.2 == 0.3)
```

Commands + Code + Text Run all

```
Fat steps: 7
SNF steps: 1
Rate per litre: 33.099999999999994
Amount: 330.99999999999994
7.000000000000002
7
7
True
False
```


Untitled8.ipynb
Saving...
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[7]
11s
text = input("Enter amount: ")
if text.replace(".", "", 1).isdigit():
amount = float(text)
print(f"Amount: {amount:8.2f}")
print(f"Integer value: {int(amount)}")
else:
print("Invalid input")
Enter amount: 250000
Amount: 250000.00
Integer value: 250000

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[8] ✓ 0s

```
RATE = 100

def rate_per_litre():
    """Return rate per litre."""
    return RATE

def payout(litres):
    """Calculate daily payout."""
    return litres * rate_per_litre()

def ten_day_total(litres):
    """Calculate total payout for ten days."""
    return payout(litres) * 10

litres = 5

print("Rate per litre:", rate_per_litre())
print("Daily payout:", payout(litres))
print("Ten day total:", ten_day_total(litres))
```

```
Rate per litre: 100
Daily payout: 500
Ten day total: 5000
```

Untitled8.ipynb ☆ Saving...

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

[9] ✓ 0s

```python
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Returns the price per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

def payout(litres, fat, snf):
    """Returns the amount payable."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:.2f} pay {pay:.2f}")
```

```
   10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00
  8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90
   12 L fat 3.1 snf 8.3 rate 27.80 pay 333.60
```

Variables Terminal

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[10] ✓ 0s

```
BASE_RATE = 30.00

total = 0

entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]

for litres, fat, snf in entries:
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE + fat_steps * 0.40 + snf_steps * 0.30
    payment = round(litres * rate, 2)
    total += payment

    print(f"Litres: {litres}, Fat: {fat}, SNF: {snf}, Rate: {rate:.2f}, Payment: {payment:.2f}")

print(f"Total payment: {total:.2f}")
```

```
Litres: 10, Fat: 3.5, SNF: 8.5, Rate: 30.00, Payment: 300.00
Litres: 8.5, Fat: 4.2, SNF: 8.7, Rate: 33.40, Payment: 283.90
Litres: 12, Fat: 3.1, SNF: 8.3, Rate: 27.80, Payment: 333.60
Total payment: 917.50
```

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[11] ✓ 0s

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
def payout(litres, fat, snf):
    if litres < 0 or fat < 0 or snf < 0:
        return 0.00
    return round(litres * rate_per_litre(fat, snf), 2)
entries = [(10, 3.5, 8.5),(8.5, 4.2, 8.7),(12, 3.1, 8.3),(5, 3.8, 8.6),(15, 3.2, 8.4)]
total = 0
for litres, fat, snf in entries:
    payment = payout(litres, fat, snf)
    total += payment
    print(f"Litres: {litres}, Fat: {fat}, SNF: {snf}, Payment: {payment:.2f}")
print(f"Total payment: {total:.2f}")
```

```
Litres: 10, Fat: 3.5, SNF: 8.5, Payment: 300.00
Litres: 8.5, Fat: 4.2, SNF: 8.7, Payment: 283.90
Litres: 12, Fat: 3.1, SNF: 8.3, Payment: 333.60
Litres: 5, Fat: 3.8, SNF: 8.6, Payment: 157.50
Litres: 15, Fat: 3.2, SNF: 8.4, Payment: 427.50
Total payment: 1502.50
```

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[12]

```
rate = 0.02

for n in [300, 6000, 150000]:
    total = 0
    for i in range(n):
        total += 0.1 * rate
    print("Payments:", n)
    print("Float Total:", total)
    print("Rounded Total:", round(n * 0.1 * rate, 2))
    print("Difference:", abs(total - round(n * 0.1 * rate, 2)))
    print()
```

```
Payments: 300
Float Total: 0.6000000000000004
Rounded Total: 0.6
Difference: 4.440892098500626e-16

Payments: 6000
Float Total: 12.000000000000677
Rounded Total: 12.0
Difference: 6.767919558114954e-13

Payments: 150000
Float Total: 300.00000000070435
Rounded Total: 300.0
Difference: 7.043468031042721e-10
```

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[13]

```
fat_rate = 0.05
snf_rate = 0.03
fat = 4.50
snf = 8.50
amount = (fat * fat_rate) + (snf * snf_rate)
print("Fat:", round(fat, 2))
print("SNF:", round(snf, 2))
print("Milk Amount:", round(amount, 2))
print("Recommended Design: Use float values and round the final amount to 2 decimal places.")
print("Rate Revision: Update the rate values when rates change.")
print("Limitation: Floating-point calculations may have small precision errors.")
print("Scalability: Suitable for larger calculations with updated rates.")
```

```
Fat: 4.5
SNF: 8.5
Milk Amount: 0.48
Recommended Design: Use float values and round the final amount to 2 decimal places.
Rate Revision: Update the rate values when rates change.
Limitation: Floating-point calculations may have small precision errors.
Scalability: Suitable for larger calculations with updated rates.
```

Untitled8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[15]

```
def calculate_payment(fat, snf, litres, season):
    base_rate = 32.00
    fat_rate = 0.45

    amount = base_rate + (fat * fat_rate)

    if season.lower() in ["april", "may", "june"]:
        pass

    bonus = 0.50 if litres > 500 else 0
    return round(amount + bonus, 2)

fat = float(input("Enter fat: "))
snf = float(input("Enter SNF: "))
litres = float(input("Enter litres: "))
season = input("Enter month: ")

print("Milk Payment:", calculate_payment(fat, snf, litres, season))
```

```
Enter fat: 4.5
Enter SNF: 8.5
Enter litres: 600
Enter month: July
Milk Payment: 34.52
```